

Entrega Contínua de uma Plataforma de Pedidos em Microsserviços

Do Docker Compose ao Kubernetes com Observabilidade e CI/CD

Autor:	Kauê Castro Oliveira	Empresa:	Loja Veloz
Função:	Cloud DevOps Engineer	Data:	Maio de 2026
Documento:	Relatório Técnico de Arquitetura e Engenharia (Parte Teórica)		

1. Conceituação de Arquitetura de Microsserviços e o Papel do DevOps

A transição de uma arquitetura monolítica tradicional para um ecossistema baseado em microsserviços constitui o alicerce fundamental para a modernização da **Loja Veloz**. Em sistemas distribuídos e *cloud-native*, as funcionalidades de negócio são fragmentadas em serviços granulares, autônomos e desacoplados: *API Gateway*, *Serviço de Pedidos*, *Serviço de Pagamentos* e *Serviço de Estoque*. Cada um desses componentes passa a deter seu próprio ciclo de vida, stack tecnológica otimizada e modelo de persistência isolado, atendendo estritamente ao princípio do contexto delimitado (*Bounded Context*).

No entanto, a descentralização introduz desafios complexos de comunicação em rede, consistência eventual de dados e sobrecarga operacional. É neste cenário que o papel do **Cloud DevOps Engineer** torna-se indispensável. Longe de ser apenas um executor de scripts, a engenharia de DevOps atua como o elo de governança e automação entre os times de desenvolvimento (Produto) e operações (SRE/Ops). Através da padronização de esteiras de entrega e da infraestrutura programável, mitiga-se o atrito técnico, permitindo que a Loja Veloz responda de forma elástica a picos de demanda — como a campanha promocional iminente — com deploys seguros e sem interrupções.

Premissa Cloud-Native: A arquitetura foi desenhada sob as diretrizes do *12-Factor App*, assegurando que as aplicações tratem configurações estritamente via variáveis de ambiente (Fator III), operem como processos *stateless* isolados (Fator VI) e exportem seus logs diretamente como fluxos de eventos contínuos (Fator XI).

2. Containerização e Isolamento: Diferenças Essenciais entre Docker e Kubernetes

Para garantir a paridade total entre o ambiente local de desenvolvimento e o cluster produtivo de alta disponibilidade, adota-se a containerização estruturada. Compreender os papéis e os limites operacionais do Docker e do Kubernetes é mandatório para o sucesso do projeto.

O **Docker** é focado no encapsulamento e na criação da unidade fundamental de implantação. Ele isola a aplicação, o runtime, as bibliotecas do sistema e as dependências necessárias dentro de uma imagem imutável. No contexto local da Loja Veloz, o **Docker Compose** atua como a ferramenta de orquestração local de múltiplos contêineres, permitindo que todo o ecossistema (APIs, banco PostgreSQL e fila de mensageria)

seja inicializado com um comando unificado. Isso elimina a recorrente inconsistência de dependências nas máquinas dos desenvolvedores.

Por outro lado, o **Kubernetes (K8s)** opera na camada acima. Ele não substitui o contêiner, mas atua como o cérebro orquestrador de múltiplos nós de computação em nível de produção. Enquanto o Docker cuida do ciclo de execução em uma única máquina, o Kubernetes realiza a governança de clusters globais, gerenciando rede distribuída, resiliência física, roteamento de tráfego externo e balanceamento de carga de maneira nativa.

Critério	Docker & Docker Compose (Local)	Kubernetes (Produção)
Escopo Principal	Empacotamento de código, isolamento local e reprodutibilidade imediata.	Orquestração em larga escala, resiliência, automação de infraestrutura.
Alta Disponibilidade	Limitado ao host local. Se o processo cai, exige intervenção manual/reinício simples.	Nativo através de <i>Self-Healing</i> . Substitui automaticamente pods que falham nas sondagens.
Escalabilidade	Manual e circunscrito aos recursos físicos da máquina de desenvolvimento.	Automático e dinâmico, baseado em métricas de uso com o <i>HPA (Horizontal Pod Autoscaler)</i> .
Casos de Uso	Ambientes de desenvolvimento, testes unitários locais e pipelines de CI integrados.	Ambientes de staging, produção corporativa e cenários de alta concorrência.

3. Fundamentação Teórica de Infraestrutura e Automação Distribuída

3.1 Orquestração de Containers em Produção

A orquestração distribuída remove a complexidade da gestão física de servidores. O Kubernetes abstrai o hardware subjacente, tratando o cluster como um pool único de recursos computacionais. Ele gerencia o ciclo de vida dos *Pods* com base em políticas declarativas. Aspectos fundamentais de produção, como o *Pod Security Admission*, garantem políticas de privilégio mínimo, assegurando que nenhum contêiner execute sob a identidade do usuário `root` do sistema operacional hospedeiro, prevenindo vetores de ataque de escape de contêiner.

3.2 Pipelines de CI/CD em Ambientes Distribuídos

Em arquiteturas de microsserviços, cada alteração em um repositório dispara uma esteira automatizada. A Integração Contínua (CI) realiza a execução compulsória de analisadores estáticos (*linters*), testes unitários e testes de integração, culminando no build de imagens enxutas através de técnicas de *multi-stage build*. A Entrega Contínua (CD) realiza a injeção automatizada e segura destes novos artefatos nos clusters, intermediando o isolamento de credenciais corporativas via injeção dinâmica de *Secrets* criptografados em tempo de execução, erradicando dados sensíveis expostos em código-fonte.

3.3 Pilares Críticos da Observabilidade Moderna

A telemetria de microsserviços difere do monitoramento reativo clássico, estruturando-se em três pilares interdependentes:

- **Métricas:** Dados numéricos agregados ao longo do tempo (como percentual de CPU, uso de memória e latência HTTP p99), fundamentais para gatilhos de alertas e auto-escalabilidade de infraestrutura.
- **Logs Structured Streams:** Linhas de eventos textuais estruturadas em formato JSON, emitidas pelas aplicações diretamente no fluxo padrão de saída (stdout) para posterior indexação centralizada e auditoria detalhada de falhas.
- **Traces Distribuídos:** Rastreamento longitudinal que injeta IDs de correlação exclusivos no cabeçalho das requisições (padrão W3C Trace Context). Permite acompanhar o caminho exato de uma transação transacionada desde o *API Gateway* até a persistência no banco ou processamento na mensageria, mapeando os pontos exatos de gargalo ou exceção técnica.

4. Justificativa Crítica das Decisões Arquiteturais Adotadas

As decisões técnicas que compõem o desenho arquitetural da plataforma foram tomadas com foco na resiliência sistêmica e na proteção das operações cruciais de varejo.

Adoção de Mensageria Assíncrona para o Fluxo de Estoque: O *Serviço de Estoque* representa o coração da operação de vendas da Loja Veloz. Gargalos no inventário, falhas em auditorias físicas ou erros no processamento e emissão de notas fiscais paralisam fluxos logísticos inteiros e geram prejuízos severos. Ao introduzir um barramento de mensageria (RabbitMQ/Kafka) para o evento *PedidoCriado*, removemos a dependência síncrona direta entre as APIs. Se o gateway aceita o pedido, o evento é postado na fila com garantia de entrega. O Serviço de Estoque processa as mensagens de forma assíncrona, respeitando sua própria capacidade de I/O, protegendo o banco PostgreSQL contra exaustão de conexões durante os minutos de pico intenso da promoção.

Estratégia de Deploy Rolling Update: Para neutralizar as indisponibilidades durante atualizações de versão, optou-se pela estratégia de *Rolling Update* nativa do Kubernetes. Esta abordagem atualiza de forma incremental as instâncias da aplicação. Novas réplicas só recebem tráfego real após a aprovação explícita das sondagens de prontidão (*Readiness Probes*). Caso um novo código injetado falhe em produção, o tráfego para a versão defeituosa é barrado na camada de roteamento, mantendo as versões íntegras em execução e permitindo um *rollback* automático e instantâneo sem qualquer impacto perceptível ao consumidor.

Escalabilidade Baseada em HPA (Horizontal Pod Autoscaler): Diante da oscilação severa prevista para a campanha promocional, determinou-se a escalabilidade horizontal dinâmica. Configurado para analisar o uso de CPU com um teto de 75% de utilização média, o HPA intermedeia o aumento dinâmico do número de pods produtivos da API de Pedidos de um mínimo de 2 instâncias operacionais até o limite elástico de 10 réplicas paralelas, garantindo vazão ao tráfego e otimizando os custos com nuvem no encerramento do evento promocional.